

# Drone-to-Satellite Image Retrieval using DualResNet

---

**Team Members:** Ahmad Sidani, Maria Al Hussien, Mariah Sandakli

## Project Overview

This project explores the task of drone-to-satellite image retrieval, aiming to match drone-captured images with their corresponding satellite images. A deep learning model DualResNet has been implemented and modified to improve retrieval accuracy by integrating and evaluating multiple enhancements such as autotuning, fine-tuning, and embedding techniques.

## 1. Model Architecture Modification

**Base Model Replaced:** The original EVA backbone was replaced with a custom DualResNet.

**Dual-Branch Processing:** One branch processes satellite images and the other handles drone images.

**Training:** Trained on a subset of the dataset and saved model weights in .pth format.

**Initial Evaluation Results:**

- Recall@1: 3.35%
- Recall@5: 12.00%
- mAP: 10.04%

## 2. Autotuning Hyperparameters

To optimize model performance, a grid search was conducted on:

- Learning Rates: [1e-3, 1e-4]
- Weight Decays: [0, 1e-5]
- Dropout Rates: [0.0, 0.3]

**Key Steps:**

- Used 100 classes with 50 drone images/class.
- Backbone weights frozen to focus on classifier layers.
- Applied data augmentations (flip, rotate, color jitter) on the fly on each drone query.
- Trained each configuration for 2 epochs and evaluated.

**Best Configuration:**

- Learning Rate = 0.01
- Weight Decay = 0

- Dropout = 0.3
- Accuracy = 41%

### 3. Fine-Tuning DualResNet

Setup:

- Model: Pretrained best\_model.pth
- Config: lr = 0.001, dropout = 0, epochs = 15, batch size = 8

Data Prep:

- Used paired satellite and drone images.
- Training data resized from 512x512 and saved to 224x224; applied standard transformations.

Architecture Modifications:

- A dropout layer with probability 0.3
- A linear layer that maps 512 input features to 100 output class scores
- Backbones frozen; only classifiers are trained.

Training:

- Loss: CrossEntropy
- Optimizer: Adam (classifier layers only)
- Backpropagation based on average branch losses.

Fine-Tuned Evaluation:

- Best Achieved Accuracy: 82.22%

### 4. Evaluation of Fine-Tuned Model

Setup:

- Only classifier layers active.
- Test folders: gallery\_satellite and query\_drone.

Feature Extraction:

- Extract normalized features using the fine-tuned model.
- View 1: Satellite, View 2: Drone

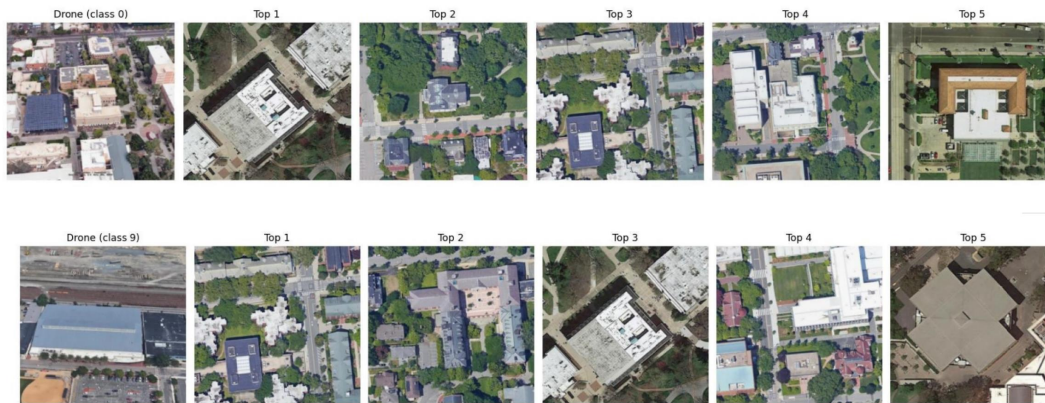
Matching & Results:

- Cosine similarity computed for retrieval.

Performance:

- Recall@1: 3.50%
- Recall@5: 13.63%
- mAP: 10.64%

## Visualization



## 5. Retrieval Enhancements: Averaged Query Embeddings

In the MBEG repository, several retrieval enhancement techniques were applied after extracting features from the backbone model. These techniques improve the quality of query embeddings and the accuracy of retrieval by processing and refining the extracted feature vectors.

### 1. Fusion Feature Extraction (Multi-Query Fusion)

Multi-query fusion averages the feature embeddings from multiple query images belonging to the same class to create a more stable and noise-resistant representation. Combining features reduces the variability caused by differences in image quality, viewpoint, or noise, resulting in more reliable query descriptors.

#### How it's implemented:

- **Feature extraction:** For each query image, extract normalized features from the backbone model separately for two different views (e.g., drone and satellite).
- **Label grouping:** Group all query features by their class labels.

- **Averaging:** Compute the mean feature vector for each class by averaging all feature vectors of queries belonging to that class. This yields one fused feature vector per class.
- **Use fused features:** Use these averaged fused vectors as the query embeddings for retrieval instead of individual query images.

#### **Performance metrics:**

- Recall@1: 4.00%
- Recall@5: 14.00%
- mAP: 11.10%

## **2. Fusion & Filtering**

This method enhances multi-query fusion by filtering the gallery images to remove irrelevant or "junk" samples before computing retrieval metrics.

By focusing only on relevant gallery images and ignoring junk data, the evaluation better reflects true retrieval performance and reduces noise from irrelevant matches.

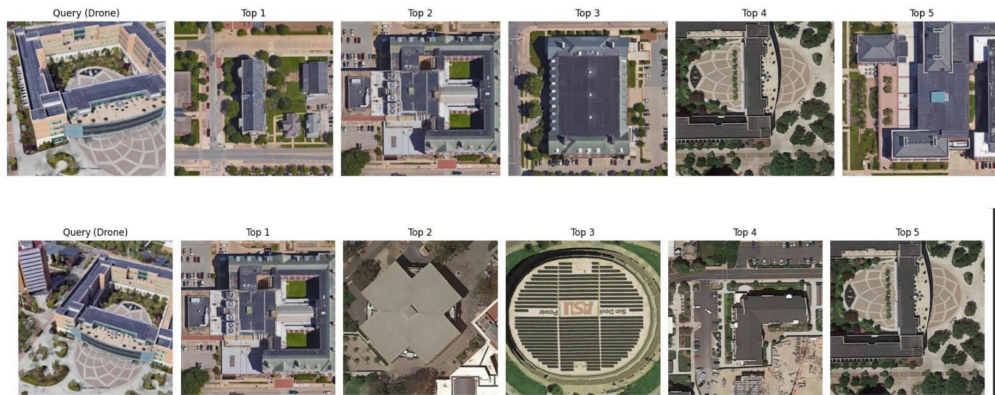
#### **How it's implemented:**

- **Multi-query fusion:** Perform the same averaging of query features by class as described above.
- **Label-based filtering:** Identify gallery images labeled as "junk" or irrelevant (for example, marked with a special label like -1).
- **Gallery filtering:** Exclude these junk images from the gallery feature set used for similarity calculation and metric computation.
- **Metric computation:** Calculate Recall@k and mAP metrics only considering the filtered gallery images, which improves the focus and reliability of evaluation.

#### **Performance metrics:**

- Recall@1: 13.93%
- Recall@5: 33.46%
- mAP: 24.38%

# Visualization



## 3. Re-ranking

Re-ranking improves initial retrieval results by considering the local similarity structure around gallery images.

Re-ranking uses the similarity of local neighborhoods within the feature space to reorder gallery matches after the initial ranking.

By considering local context (neighbors' similarities), re-ranking aims to boost relevant images that may have been ranked lower initially.

### How it's implemented:

- **Initial retrieval:** Compute cosine similarity scores between each query and all gallery satellite embeddings, resulting in a full ranked list of gallery images for each query.
- **Neighborhood calculation:** For each gallery image, find its closest neighbors in feature space.
- **Re-ranking process:** Adjust the rank of gallery images based on how similar their neighbors are to the query. For example, if a gallery image has many neighbors close to the query, its rank improves.
- **Final rank list:** Produce an updated, re-ordered list of gallery images for each query.
- **Metric evaluation:** Use the re-ranked list to compute retrieval metrics.

### Performance metrics:

- Recall@1: 0.02%
- Recall@5: 4.78%
- mAP: 5.68%

*Note:* In this case, re-ranking yielded low performance, possibly due to dataset characteristics or parameter settings.

## 4. Whitening

Whitening transforms the feature vectors to have zero mean and identity covariance, removing correlation between feature dimensions.

This normalization reduces redundancy and improves the discriminative power of embeddings by standardizing their distribution.

### How it's implemented:

- **Compute mean and covariance:** Calculate the mean vector and covariance matrix of the gallery feature vectors.
- **Whitening matrix:** Derive a whitening matrix from the covariance matrix (e.g., via eigen-decomposition or PCA whitening).
- **Apply transform:** Subtract the mean from each feature vector and multiply by the whitening matrix to obtain whitened features.
- **Apply same transform:** Apply the whitening transformation to both query and gallery feature sets before similarity computation.
- **Metric evaluation:** Compute Recall@k and mAP using whitened features.

### Performance metrics:

- Recall@1: 1.5%
- Recall@5: 4.78%
- mAP: 5.68%

## 5. Test Time Augmentation (TTA)

TTA involves augmenting images during inference and averaging the resulting feature vectors to create more robust embeddings.

Augmentation (such as horizontal flips) accounts for variability in image orientation or appearance, resulting in embeddings less sensitive to these factors.

### How it's implemented:

- **Feature extraction from original:** Extract normalized features from the original query and gallery images.
- **Feature extraction from augmented images:** Apply a horizontal flip to the images and extract features from these flipped versions.
- **Feature averaging:** Average the features from the original and flipped images to produce a final, robust embedding for each image.
- **Fusion:** Concatenate or fuse features from two different views (e.g., drone and satellite) to enhance representation.
- **Filtering:** Remove junk images from the gallery to focus on relevant matches.
- **Metric evaluation:** Compute retrieval metrics with the enhanced embeddings.

### Final Performance:

- Recall@1: 15.22%
- Recall@5: 34.76%
- mAP: 25.80%

## 6. Finding TOP 1 Match

In an attempt to improve retrieval performance, we experimented with the following strategies:

### 1. Top-1 Matching Per Class

We modified the retrieval logic so that each query drone image was matched only to the top-ranked gallery image **per class**, rather than across the entire gallery. This was intended to reduce noise and balance the retrieval across classes.

- **Observation:**  
Despite the intuition, this approach **did not improve accuracy**. In fact, results degraded, likely due to forced selection from less relevant classes, which diluted the effect of true nearest neighbors.

### 2. Top-1 Matching Per Query

We also evaluated performance by selecting only the top-1 match for **each individual query drone image**, instead of using fusion or averaging.

- **Observation:**

This method also **did not improve** retrieval metrics. The absence of multi-query fusion led to noisier representations, and performance was consistently lower than fusion-based results.

Note: The visualizations are added to the github.

## 7. Codalab Challenge Submission

Steps for Challenge Submission:

- Loaded and evaluated the fine-tuned DualResNet model.
- Applied preprocessing (resize, RGB, normalize).
- Extracted and normalized fused features from both model branches.
- Calculated cosine similarity matrix between queries and gallery images.
- Top 10 matches per query were extracted and output for submission.